

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Лабораторная работа № 4
По СПП
Вариант 4

Выполнила:
Студент 3 курса
Группы ПО-12
Езерская О.Г.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL.

4. Анализ популярных технологий в открытых репозиториях GitHub

Условие:

Напишите Python-скрипт, который:

1. Запрашивает у пользователя ключевое слово и тему (например, "machine learning", "web development", "blockchain").

Использует GitHub API для поиска 100+ самых популярных репозиторий по этому ключевому слову.

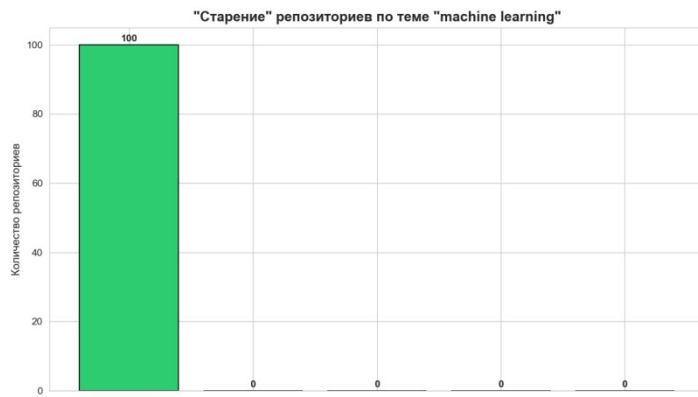
3. Для каждого найденного репозитория собирает:

- ☐ Язык программирования
- ☐ Количество звезд
- ☐ Количество форков
- ☐ Количество открытых issues
- ☐ Дату последнего обновления

4. Анализирует какие технологии чаще всего используются в данной области, строя рейтинг языков программирования.

5. Визуализирует результаты:

- ☐ Диаграмму популярных языков программирования (matplotlib, seaborn, plotly)
- ☐ График популярности репозиторий (по звёздам и активности)
- ☐ График "старения" репозиторий (когда последний коммит).



```
# pylint: disable=invalid-name, line-too-long
```

```
"""Модуль для анализа популярных репозиторияв GitHub."""
```

```
import sys
```

```
from collections import Counter
```

```
from datetime import datetime, timezone
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
import requests
```

```
import seaborn as sns
```

```
sns.set_style("whitegrid")
```

```
plt.rcParams["font.size"] = 10
```

```
def search_github_repos(keyword, per_page=100):
```

```
    """Поиск 100+ самых популярных репозиторияв по ключевому слову."""
```

```
    repos_data = []
```

```
    headers = {"Accept": "application/vnd.github.v3+json"}
```

```
    params = {"q": keyword, "sort": "stars", "order": "desc", "per_page": min(per_page,
100)}
```

```
    print("Запрашиваем данные с GitHub API...")
```

```

try:
    response = requests.get(
        "https://api.github.com/search/repositories", headers=headers, params=params,
        timeout=30
    )
except requests.exceptions.RequestException as e:
    print(f"Ошибка подключения: {e}")
    return []

if response.status_code != 200:
    print(f"Ошибка API: {response.status_code}")
    print(response.json().get("message", "Неизвестная ошибка"))
    return []

data = response.json()
print(f"Найдено всего репозитория: {data['total_count']}")

for item in data["items"][:per_page]:
    repos_data.append(
        {
            "name": item["full_name"],
            "language": item["language"],
            "stars": item["stargazers_count"],
            "forks": item["forks_count"],
            "open_issues": item["open_issues_count"],
            "last_updated": item["updated_at"],
        }
    )
return repos_data

```

```
def analyze_languages(repos_data):
    """Анализирует какие технологии чаще всего используются."""
    languages = [repo["language"] for repo in repos_data if repo["language"]]
    lang_counts = Counter(languages)
    total = len(languages)
    sorted_langs = lang_counts.most_common()
    top_langs = sorted_langs[:4]
    other_count = sum(count for _, count in sorted_langs[4:])
    if other_count > 0:
        top_langs.append(("Другие", other_count))
    return top_langs, total
```

```
def plot_language_diagram(lang_data, keyword):
    """Диаграмма популярных языков программирования."""
    labels, sizes = zip(*lang_data)
    plt.figure(figsize=(8, 8))
    plt.pie(sizes, labels=labels, autopct="%1.1f%%", startangle=90,
            colors=sns.color_palette("husl", len(labels)))
    plt.title(f'Популярные языки в теме "{keyword}"', fontsize=14, fontweight="bold")
    plt.axis("equal")
    plt.tight_layout()
    filename = f'{keyword.replace(" ", "_")}_languages.png'
    plt.savefig(filename, dpi=150, bbox_inches="tight")
    plt.show()
    print(f'Диаграмма языков сохранена в '{filename}''')
```

```
def plot_popularity_chart(repos_data, keyword):
    """График популярности репозитория (по звёздам и форкам)."""
    top_repos = repos_data[:20]
```

```

names = [repo["name"].split("/")[-1][:20] for repo in top_repos]
stars = [repo["stars"] for repo in top_repos]
forks = [repo["forks"] for repo in top_repos]

x = np.arange(len(names))
_, ax = plt.subplots(figsize=(12, 7))
ax.bar(x - 0.175, stars, 0.35, label="Звёзды", color="gold", alpha=0.8)
ax.bar(x + 0.175, forks, 0.35, label="Форки", color="steelblue", alpha=0.8)
ax.set_xlabel("Репозитории", fontsize=11)
ax.set_ylabel("Количество", fontsize=11)
ax.set_title(f'Популярность по теме "{keyword}"', fontsize=14, fontweight="bold")
ax.set_xticks(x)
ax.set_xticklabels(names, rotation=45, ha="right", fontsize=8)
ax.legend()
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
filename = f'{keyword.replace(" ", "_")}_popularity.png'
plt.savefig(filename, dpi=150, bbox_inches="tight")
plt.show()
print(f'График популярности сохранен в '{filename}''')

```

```

def plot_aging_chart(repos_data, keyword):
    """График старения репозитория."""
    now = datetime.now(timezone.utc)
    ages = []
    for repo in repos_data:
        last = datetime.fromisoformat(repo["last_updated"].replace("Z", "+00:00"))
        ages.append((now - last).days)

    ranges = [(0, 30), (31, 90), (91, 180), (181, 365), (366, float("inf"))]

```

```

labels = [
    "Обновлялись\nпоследний месяц",
    "Обновлялись\n3 месяца назад",
    "Обновлялись\n6 месяцев назад",
    "Обновлялись\nгод назад",
    "Не обновлялись\nбольше года",
]

counts = [sum(1 for age in ages if low <= age <= high) for low, high in ranges]

plt.figure(figsize=(10, 6))

chart_bar = plt.bar(
    labels, counts, color=["#2ecc71", "#f39c12", "#e67e22", "#e74c3c", "#95a5a6"],
    edgecolor="black", linewidth=1
)

plt.xlabel("Возраст репозитория", fontsize=11)
plt.ylabel("Количество", fontsize=11)
plt.title(f"Старение" по теме "{keyword}", fontsize=14, fontweight="bold")
plt.xticks(rotation=0, ha="center", fontsize=9)
for single_bar, count in zip(chart_bar, counts):
    plt.text(
        single_bar.get_x() + single_bar.get_width() / 2,
        single_bar.get_height() + 0.5,
        f"{count}",
        ha="center",
        va="bottom",
        fontsize=10,
        fontweight="bold",
    )

plt.tight_layout()

filename = f'{keyword.replace(" ", "_")}_aging.png'
plt.savefig(filename, dpi=150, bbox_inches="tight")

```



```
plt.show()

print(f"График старения сохранен в '{filename}'")

old = sum(1 for age in ages if age > 365)

print(f"\n{old / len(ages) * 100:.0f}% репозитории не обновлялись больше года!")
```

```
def main():

    """Основная функция."""

    keyword = input("Введите тему для анализа: ").strip()

    if not keyword:

        print("Тема не может быть пустой!")

        sys.exit(1)

    print(f"\nАнализируем 100 популярных репозитории по теме \"{keyword}\"...")

    repos_data = search_github_repos(keyword, per_page=100)

    if not repos_data:

        print("Не удалось получить данные.")

        sys.exit(1)

    print(f"Получено {len(repos_data)} репозитории\n")

    lang_stats, total = analyze_languages(repos_data)

    print("Самые популярные языки:")

    for lang, count in lang_stats:

        print(f"- {lang} ({count / total * 100:.0f}%)")

    most_starred = max(repos_data, key=lambda x: x["stars"])

    print(f"\nСамый звёздный: \"{most_starred[\"name\"]}\" ({most_starred[\"stars\"]:,} звёзд)")

    avg_forks = sum(r["forks"] for r in repos_data) / len(repos_data)
```

```
print(  
    f"Среднее количество форков: {avg_forks:.1f}k"  
    if avg_forks >= 1000  
    else f"Среднее количество форков: {avg_forks:.0f}"  
)
```

```
plot_language_diagram(lang_stats, keyword)  
plot_popularity_chart(repos_data, keyword)  
plot_aging_chart(repos_data, keyword)
```

```
if __name__ == "__main__":  
    main()
```

Вывод: научились использовать Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL.